Université Hassan II Casablanca
Faculté des Sciences Ben M'Sik
Département de Mathématiques et Informatique

PROJET FIN DE MODULE MASTER IA

Infrastructure DevOps Avec CI/CD Kubernetes

Under the supervision of :			Presented by :
Pr.Fouzia	Benabbou	Encadrante	Malak Houali Hiba Dellaji Othman Salahi CHABBAKI Ayman

Academic Year: 2025/2026

SOMMAIRE

Projet context 1

Conteneurisation 2

Terraform 3

Ansible 4

Kubernetes 5

CI/CD & Jenkins 6

- **Pourquoi ce projet?**
  - Le développement moderne nécessite des applications non seulement fonctionnelles, mais aussi fiables, automatisées et faciles à déployer dans des environnements cloud.
  - L'approche DevOps permet de réduire les erreurs et d'accélérer les mises en production grâce à l'automatisation.
- **Notre approche**
  - Nous utilisons les pratiques DevOps et CI/CD pour automatiser tout le cycle de vie de l'application : tests, conteneurisation avec Docker, déploiement avec Kubernetes, et gestion de l'infrastructure avec Terraform et Ansible.

- AI Voice Interview est une plateforme web intelligente qui automatise les entretiens d'embauche via la voix.
- Elle analyse les réponses des candidats et génère une évaluation avec un score.

01

Fonctionnement de l'IA

Le système permet :

- une interaction vocale avec le candidat
- une analyse en temps réel avec NLP (Sentence-BERT, PyTorch)
- une génération automatique des résultats

02

Architecture de l'application

- Backend
  API REST avec FastAPI
  Intégration des modèles d'IA
- Frontend
  Interface en React / Vite
  Interaction fluide en temps réel
- Base de données
  PostgreSQL
  Stockage des données (candidats, sessions, résultats)

03

Objectif DevOps

- Déploiement sur Microsoft Azure
- Automatisation avec CI/CD
- Gestion et mise à jour continues de l'application
-

Objectifs du Projet

Projet context & Conteneurisation | Terraform & Ansible | Kubernetes | CI/CD & Docker

1. Conteneurisation & Application

- Dockerisation du backend et frontend
- Optimisation avec multi-stage builds

2. Infrastructure Cloud

- Provisionnement sur Microsoft Azure avec Terraform
- Création des VM, réseau, sécurité et IP

3. Configuration & Orchestration

- Configuration automatique avec Ansible
- Mise en place du cluster Kubernetes
- Déploiement via fichiers YAML (Deployments, Services...)

4. CI/CD & Sécurité

- Pipelines Jenkins (infrastructure + application)
- Déploiement automatique après chaque push
- Application des bonnes pratiques de sécurité

Couche Applicative : Conteneurisation avec Docker

1. Objectifs clés

- Garantir la reproductibilité totale entre les environnements (Dev, Test, Prod).
- Isolation complète des services : Backend (FastAPI), Frontend (React/Vite) et Base de données (PostgreSQL).

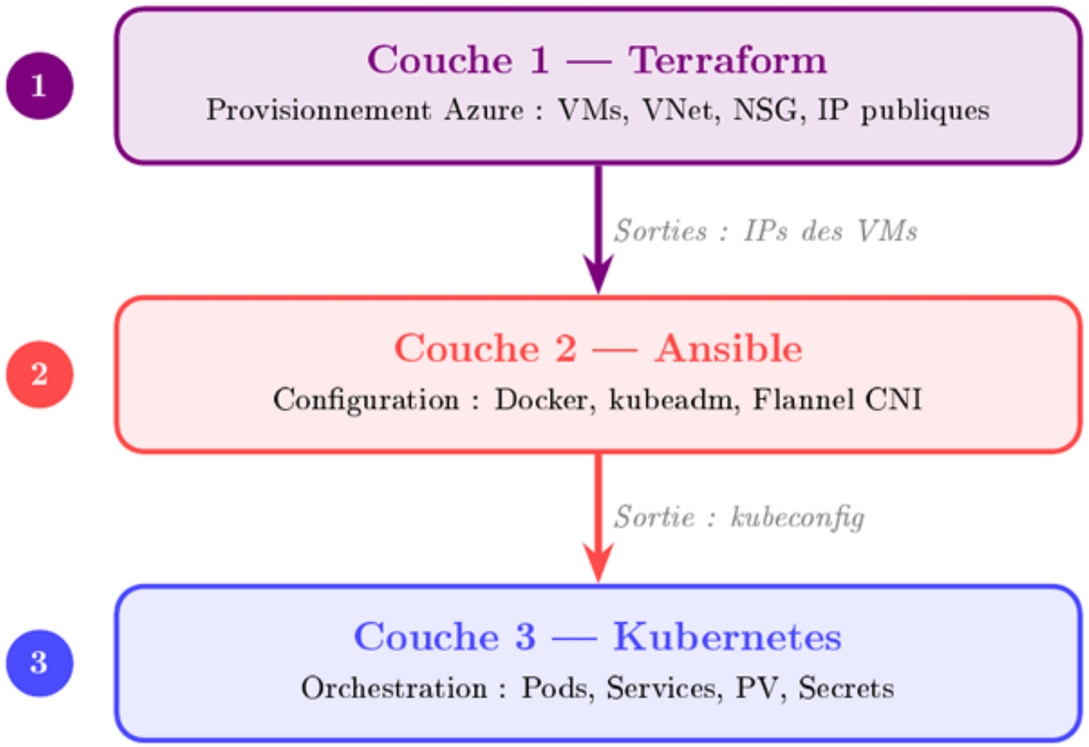
2. Optimisation

- Séparation de l'environnement de build et d'exécution pour réduire la surface d'attaque.
- Résultat : Image Backend réduite de 1.2 GB à environ 280 MB (utilisation d'images slim et alpine).

3. Stratégie de Tagging

- Utilisation de 3 tags par image : :latest, :build-N (rollback) et :commit-hash (lien direct au code).

L'infrastructure du projet est organisée en trois couches complémentaires, assurant une automatisation complète et une bonne séparation des responsabilités.



→ Création des ressources cloud sur Microsoft Azure (machines virtuelles, réseau, sécurité)

→ Installation et configuration automatique de Docker et Kubernetes

→ Déploiement et gestion des conteneurs (backend, frontend, base de données)

Terraform : Infrastructure as Code (IaC)

Provisionnement Cloud

- Automatisation Azure : Déploiement de l'infrastructure via fichiers HCL.
- Gestion d'état : Suivi des ressources via terraform.tfstate.
- Idempotence : Garantie d'un environnement reproductible à chaque run

Instances Déployées (Ubuntu 22.04 LTS) :

- **1 Nœud Maître (Master) :** Héberge le plan de contrôle Kubernetes (API server, etcd).
- **1 Nœud Worker :** Exécute les pods applicatifs (Backend, Frontend, PostgreSQL).
- **1 Serveur CI/CD :** Support pour Jenkins, Terraform et Ansible.

Composants Réseau & Sécurité

- Isolation Réseau : Création d'un VNet (10.0.0.0/16) et Subnet dédié.
- Filtrage Flux (NSG) : Ouverture des ports critiques (SSH, API K8s, NodePorts).
- Bastion SSH : Accès aux workers via Proxy Jump pour plus de sécurité.

Ancible : Infrastructure as Code (IaC)

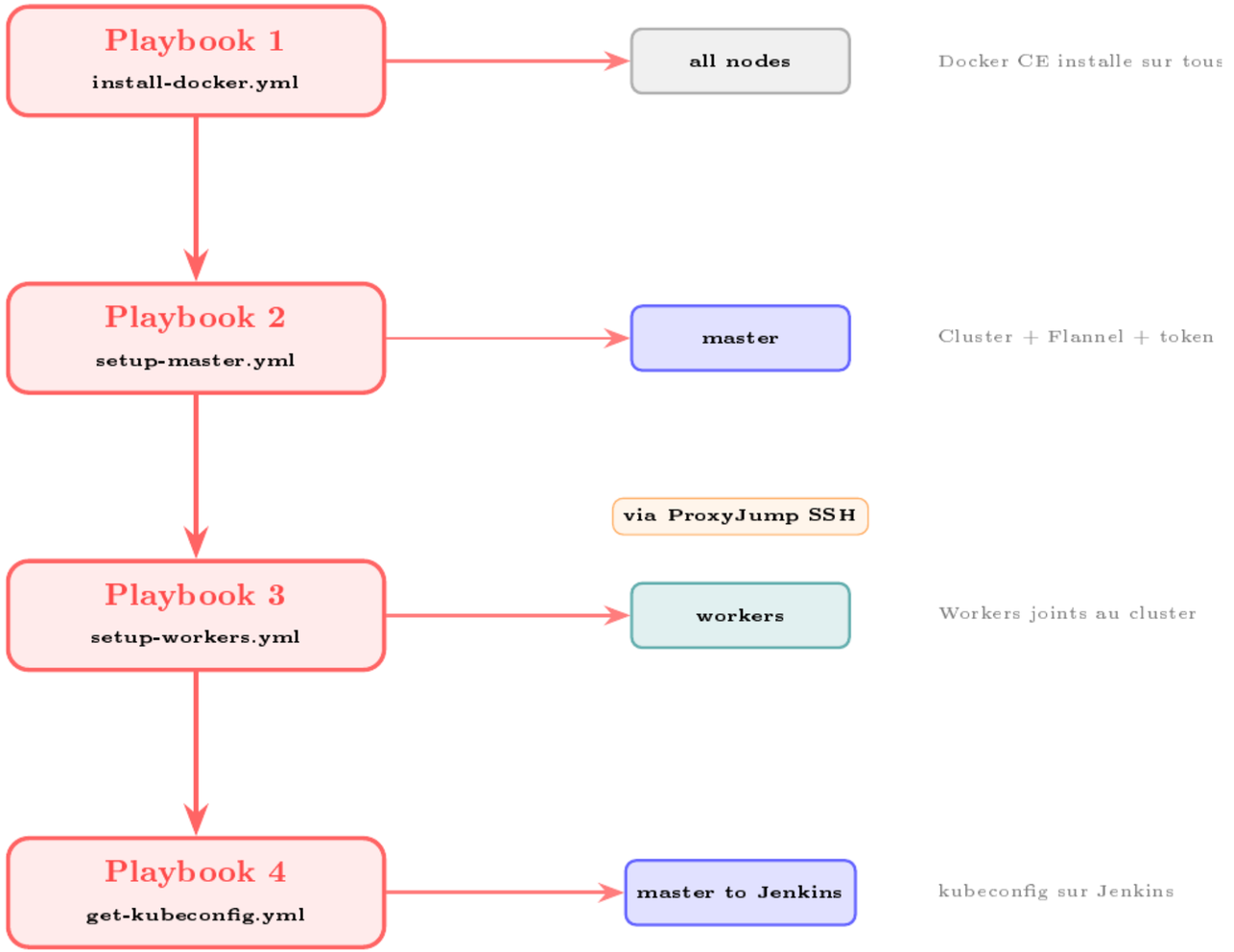
Gestion de Configuration "Agentless"

- Communication sécurisée via SSH avec exécution de playbooks YAML idempotents.
- Inventaire Dynamique : Les adresses IP des VMs sont extraites directement des outputs de Terraform par Jenkins pour alimenter le fichier hosts.ini.

Flux d'Automatisation (4 Playbooks)

1. **Installation Docker CE :** Déploiement du moteur de conteneurs sur tous les nœuds.
2. **Master Node :** Initialisation du cluster avec kubeadm et mise en place de Flannel (CNI).
3. **Worker Nodes :** Intégration au cluster via le token généré (Proxy Jump via le Master).
4. **Kubeconfig :** Transfert vers Jenkins pour la gestion du cluster.

Ordre d'exécution des playbooks Ansible et leurs cibles



Orchestration du Cluster : Kubernetes (K8s)

Isolation & Sécurité

- Déploiement dans un **Namespace dédié (ai-interview)** pour l'isolation logique des ressources
- Gestion des informations sensibles (mots de passe DB, clés JWT) via les Kubernetes **Secrets**

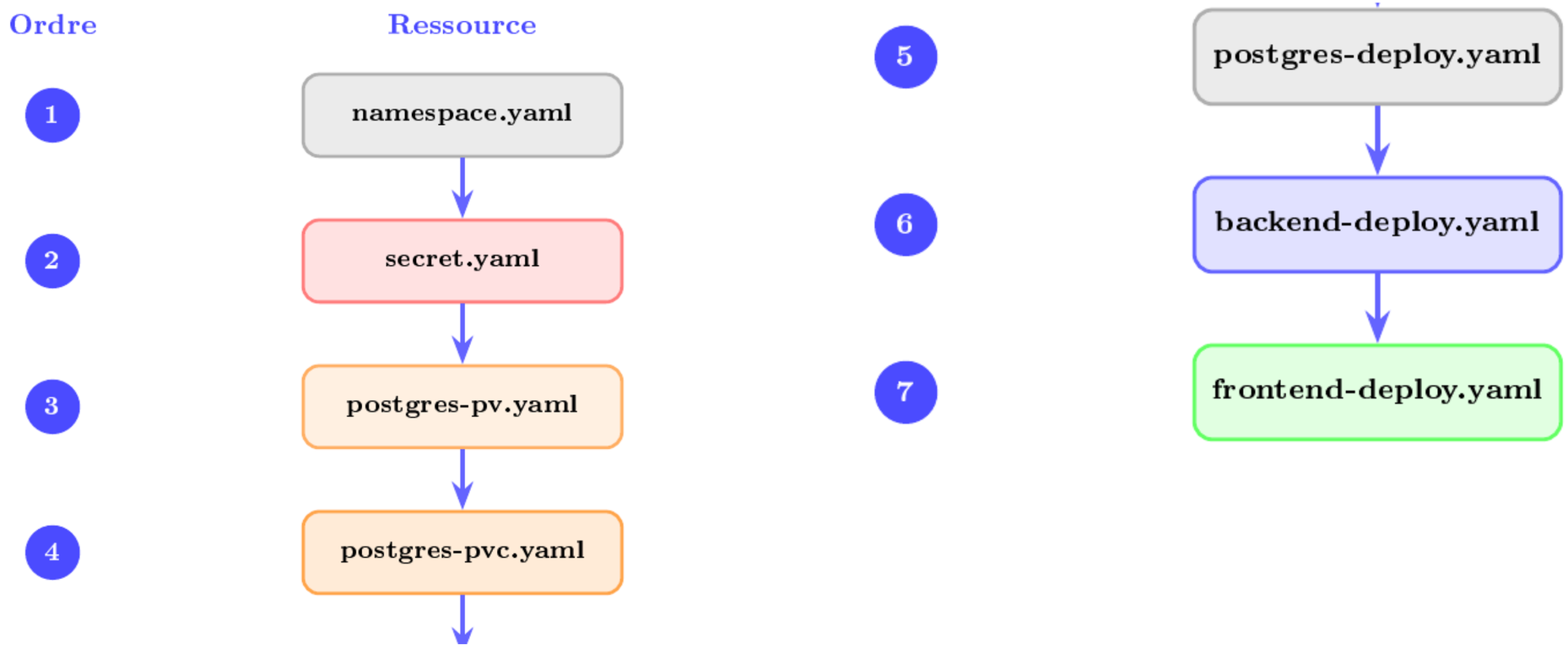
Auto-healing

- Vérification de l'état de santé via des **Liveness** et Readiness probes sur l'endpoint **/health**

Persistance & Disponibilité

- **PostgreSQL**: Utilisation de **Persistent Volumes** (PV) et **Claims** (PVC) pour garantir la survie des données après redémarrage des pods
- **Services** de type **NodePort** pour l'exposition externe sur Azure:
  - NodePort (30080) pour le Frontend
  - NodePort (30800) pour le Backend

Ordre d'application des manifestes Kubernetes



Validation Automatisée du Code (Phase CI)

La phase d'Intégration Continue garantit la qualité et la sécurité du code avant tout déploiement, en exécutant automatiquement une série de validations à chaque modification.

1. **Étape 1 — Déclenchement**
   Détection des modifications

   Démarre le workflow

2. **Étape 2 — Automatisation**
   Exécution des instructions

   Exécute les vérifications

3. **Étape 3 — Validation**
   Contrôle du fonctionnement

   Analyse la conformité

4. **Étape 4 — Sécurité**
   Prévention des vulnérabilités

Déclenchement instantané : Webhook GitHub activé à chaque "Git Push" pour un feedback en temps réel.

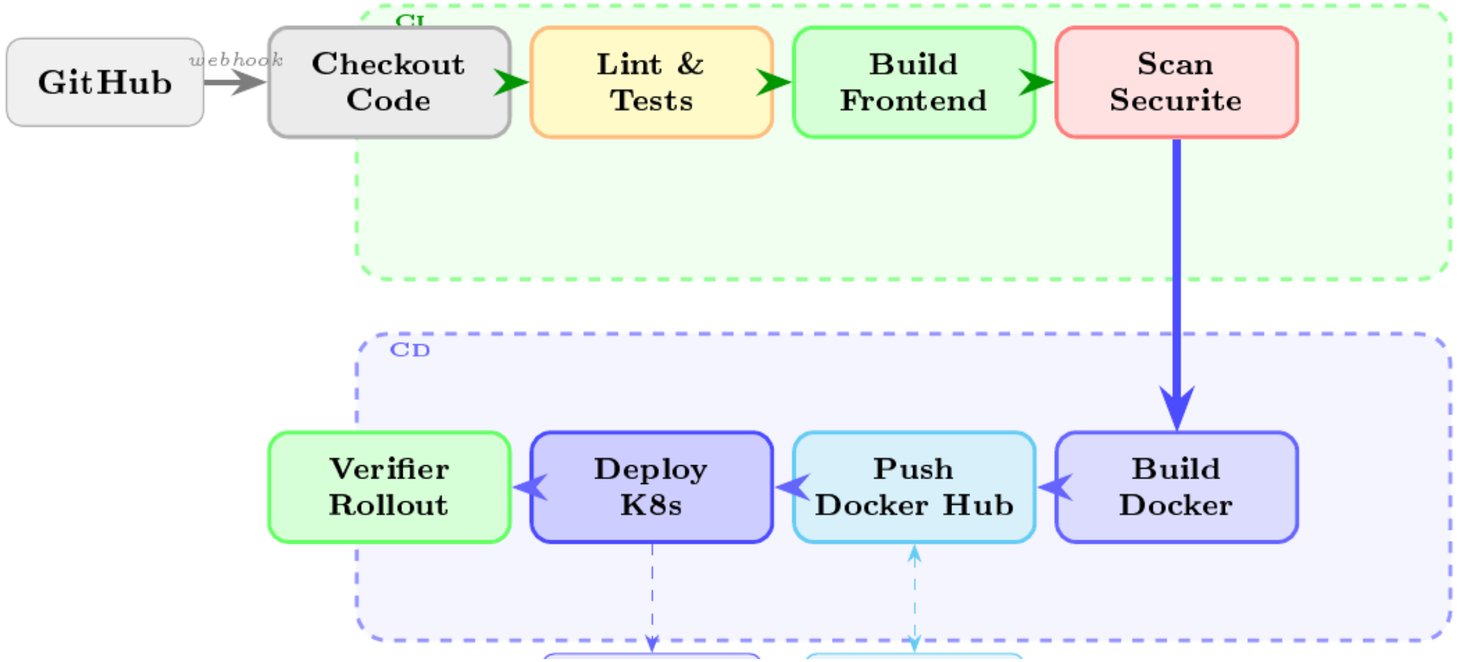
Automatisation complète : Pipeline as Code via un fichier Jenkinsfile versionné sur GitHub.

Validation fonctionnelle : Exécution systématique des tests unitaires (Pytest Backend, Vitest Frontend).

Qualité & Sécurité : Scans de vulnérabilités (Safety CVE) et analyse statique du code (Flake8, Black).

Les stages utilisé dans CI/CD pipeline:

La phase d'Intégration Continue garantit la qualité et la sécurité du code avant tout déploiement, en exécutant automatiquement une série de validations à chaque modification.



Le pipeline est composé de 11 étapes (stages) exécutées séquentiellement

Phase CI (Intégration Continue)

- Checkout Code
- Lint & Tests
- Build Frontend
- Scan Sécurité

Phase CD (Déploiement Continu)

- Build Docker
- Push Docker Hub
- Deploy K8s
- Vérifier Rollout

Packaging et Livraison Continue (Phase CD)

Conteneurisation

Build multi-étapes optimisé des images Docker pour réduire la taille et la surface d'attaque en production.

Traçabilité

Stratégie de tagging triple : Build ID (rollback), Hash Git (code source) et "Latest" pour Kubernetes.

Déploiement

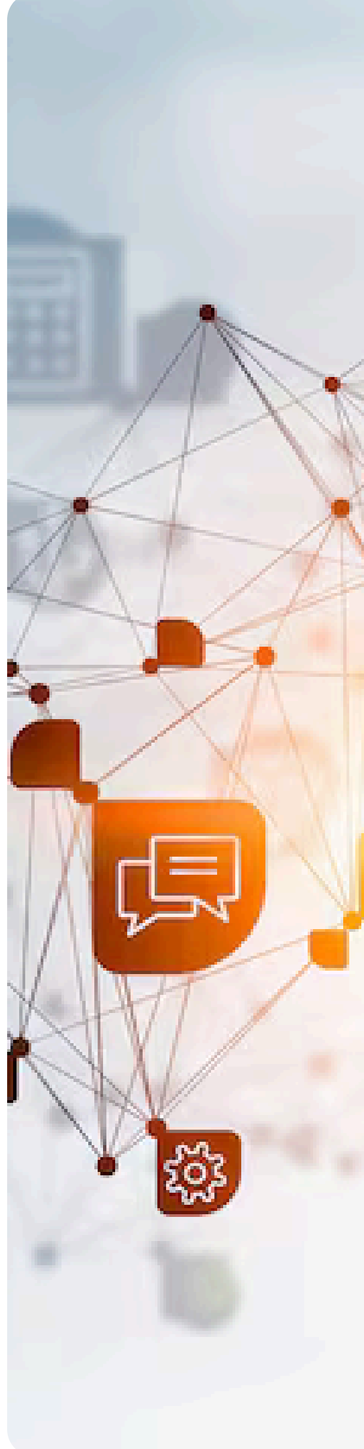
Envoi sécurisé vers Docker Hub et mise à jour automatique des pods sur le cluster Kubernetes Azure.

Point technique : L'isolation des builds est garantie par l'utilisation de variables dynamiques basées sur le numéro de build Jenkins, évitant tout conflit de ressources.

Conclusion

L'approche DevOps appliquée au projet **AI Voice Interview** a permis de transformer un concept innovant en une solution cloud fiable, automatisée et évolutive.

- Infrastructure as Code : Déploiement rapide et standardisé sur Microsoft Azure grâce à la synergie de Terraform et Ansible.
- Haute Disponibilité : Orchestration robuste, sécurisée et scalable de nos microservices via Kubernetes.
- Livraison Continue : Pipelines CI/CD automatisés garantissant la qualité du code et la fluidité des mises en production.



MERCI

pour votre attention